

Card Controller Pro

Documentation & Script Reference

UNITY 6 · URP 17.3 · DOTWEEN · INPUT SYSTEM

Table of Contents

1. [Requirements](#)
2. [Quick Start](#)
 1. Set Up a Card Prefab
 2. Create a CardData Asset
 3. Set Up a CardGroup
 4. Populate Cards
 5. Set Up Drop Zones
3. [Card Types](#)
4. [Layout Options](#)
5. [Animations](#)
 1. Place Animations
 2. Attack Animations
 3. Hover Animations
 4. Pickup Animations
 5. Select Animations
 6. Use Animations
 7. Hide Animations
6. [Adding and Removing Cards](#)
7. [Click Actions](#)
8. [Card Selection](#)
9. [Card Flipping](#)
10. [Drag-and-Drop Between Groups](#)
11. [Reordering Cards During Drag](#)
12. [Targeting Arrow](#)
13. [Responding to Card Events](#)
14. [Card Health](#)
15. [Dealing Cards with Delay](#)
16. [Tilt Effect](#)
17. [Drag Spring](#)
18. [Global Settings](#)
19. [Creating Custom Animations](#)
20. [Script Reference](#)
 1. Card

- 2. CardGroup
- 3. CardData
- 4. CardDropZone
- 5. CardGroupInitializer
- 6. CardHealth
- 7. CardControllerSettings
- 8. Animation Interfaces

21. [Creating an Assembly Definition](#)

22. [Included Prefabs](#)

23. [Demo Scenes](#)

1. Requirements

- Unity 6 or later
- DOTween or DOTween Pro (Demigiant)
- Unity Input System

2. Quick Start

2.1. Set Up a Card Prefab

Use the included `Card.prefab` or create your own. The easiest approach is to duplicate `Card.prefab` and then modify it to your liking. The card requires the following GameObject hierarchy:

```
Card                ← RectTransform + CanvasGroup + Card component
  cardTilter         ← idle/hover tilt
    cardDragRotator  ← spring wobble during drag
      cardFlipper    ← Y-axis flip rotation
        cardPickuper ← pickup animation target
          cardHoverer ← hover animation target
            cardVisuals
              front    ← front face (Image)
              back     ← back face (Image)
              border   ← targeting highlight (hidden by default)
            selectionMark ← selection indicator (hidden by default)
          DynamicArrow ← dotted arrow for targeting (optional)
```

Each layer in the hierarchy is animated independently so the effects do not interfere with each other. Assign each GameObject to the matching field on the `Card` component in the Inspector.

2.2. Create a CardData Asset

Right-click in the Project window and choose **Create > CardControllerPro > Card Data**.

Configure the following fields:

- **Card Name / Description** – display information shown on the card.
- **Front Art / Back Art** – sprites for the card faces.
- **Card Type** – determines how the card behaves when dragged (see [Section 3](#)).
- **Animations** – choose a Place, Attack, Pickup, and Use animation from the dropdowns.

2.3. Set Up a CardGroup

Add the `CardGroup` component to an empty GameObject inside your Canvas. A CardGroup represents a logical zone for cards — a hand, a deck, a field zone, and so on.

Key settings:

- **Card Spacing** – horizontal distance between cards.
- **Fan Radius** – set to `0` for a straight line, or a value such as `800` for a curved fan. Positive values curve downward; negative values curve upward.
- **Card Scale** – uniform scale applied to every card in the group.
- **Allow Pickup** – whether cards can be dragged out of this group.

2.4. Populate Cards

Option A – CardGroupInitializer (recommended): Add the `CardGroupInitializer` component to any `GameObject`. In the Inspector, add entries specifying which prefab, `CardData` asset, and count to spawn into which `CardGroup`. The component runs automatically on `Start`.

Option B – Code:

```
GameObject cardObj = Instantiate(cardPrefab);
Card card = cardObj.GetComponent<Card>();
card.Initialize(myCardData);
myCardGroup.AddBack(card);
```

2.5. Set Up Drop Zones (optional)

To allow players to drag cards from one group to another, add a `CardDropZone` component to any UI element (for example, an `Image`). Assign the target `CardGroup` to the drop zone's `cardGroup` field. The drop zone automatically appears when a card is being dragged and hides otherwise.

3. Card Types

Set the card type on the `CardData` asset. This controls what happens when the player drags the card.

Type	Drag Behavior
Place	Card follows the mouse. Drop on a <code>CardDropZone</code> to move it to another group.
Attack	Card stays in place. A targeting arrow appears. Release over a valid target to trigger an attack animation.
Use	Card follows the mouse. Drop on a <code>CardDropZone</code> to play a use animation, then the card is destroyed.
TargettedUse	Targeting arrow (like Attack). Release over a target to play a use animation on that target, then the card is destroyed.
Static	Card cannot be dragged at all.

3.1. Placeable From Hand

The `placeableFromHand` toggle on `CardData` (default: `true`) changes how Attack cards behave in the player's hand. When enabled, Attack cards dragged from a group marked `isPlayerHand` will behave as Place cards — allowing players to drag attack cards onto the board, where they then function as attack cards.

4. Layout Options

All layout is configured per `CardGroup`.

- **Straight line** (default): Set `fanRadius = 0`. Cards are spaced evenly using `cardSpacing`.
- **Fan / arc**: Set `fanRadius` to a non-zero value (for example, `800`). Positive values curve downward; negative values curve upward. `cardSpacing` controls how spread out the fan is.
- **Stacked / deck**: Set `cardSpacing = 0` and use `yOffsetPerCard` and/or `zOffsetPerCard` for slight visual offsets. Enable `anchorToFirstCard` to stack from the first card's position instead of centering.

5. Animations

Animations are configured through Inspector dropdowns. Each dropdown lists all available options for that animation type.

5.1. Place Animations

Configured on `CardData`. Plays when a card is placed onto a drop zone.

Animation	Description
Default	No animation; instant snap.
Instant	Kills active tweens and snaps to position.
Swirl	360-degree rotation with a scale pulse.
Linear Slide	Smooth linear slide to the destination.
Fade In	Fades in from transparent.
From Below	Scales up from zero with a shake.
Custom Curve	Drive position and scale with your own <code>AnimationCurve</code> s.
Custom DOTween	Separate DOTween <code>Ease</code> settings for position and scale.

5.2. Attack Animations

Configured on `CardData`. Plays when a card attacks a target.

Animation	Description
Arc	Curved Bézier path to the target with a scale punch on impact.
Quick Dash	Fast linear dash to the target and back.
Fade Teleport	Fade out, appear at target, fade back in.
Impact Strike	Tilt and scale punch in place (no movement).
Custom Curve	<code>AnimationCurve</code> control for the approach and return phases.
Custom DOTween	DOTween <code>Ease</code> control for the approach and return phases.

5.3. Hover Animations

Configured on **CardGroup**. Plays when a card is hovered.

Animation	Description
None	No visual effect.
Scale Up	Scales the card up.
Punch Scale	Quick punch-scale pop.
Shake Rotation 2D	Z-axis wiggle.
Shake Rotation 3D	3D rotation shake.

The `CardGroup` also handles the positional hover effect (lifting the card and pushing neighbors aside), controlled by `hoverYOffset`, `hoverZOffset`, `makeRoomOnHover`, and `hoverSpacing`.

5.4. Pickup Animations

Configured on **CardData**. Plays when a card is clicked or pressed.

Animation	Description
Default	No visual effect.
Scale Up	Scales the card up while held.
Punch Scale	Quick punch pop on pickup.
Shake Rotation 2D	Z-axis wiggle on pickup.
Shake Rotation 3D	3D rotation shake on pickup.
Transparency	Fades the card to semi-transparent while held.

5.5. Select Animations

Configured on **CardGroup**. Plays when a card is selected (requires `onClickAction = Select`).

Animation	Description
None	No visual effect.
Scale	Scales the card up while selected.
Lift	Lifts the card upward while selected.

5.6. Use Animations

Configured on **CardData**. Plays when a Use or TargettedUse card is activated.

Animation	Description
None	No animation; completes immediately.
Dash	Dashes toward the target, or shrinks if no target.
Shake	Shakes before completing.

5.7. Hide Animations

Hide animations are not configured in the Inspector. Instead, pass them as a parameter when removing cards from a group:

```
cardGroup.RemoveFront(new FadeOutHideAnimation(), (card) => {
    Destroy(card.gameObject);
});
```

Animation	Description
FadeOut	Fades the card to transparent.
Shrink	Scales the card to zero with a shake.

6. Adding and Removing Cards

6.1. Adding Cards

```
cardGroup.AddBack(card);           // add to end
cardGroup.AddFront(card);          // add to front
cardGroup.AddAtPosition(card, 2);   // add at index

// With a place animation override:
cardGroup.AddBack(card, new LinearSlidePlaceAnimation());
```

6.2. Removing Cards

```
Card card = cardGroup.RemoveFront();           // remove first
Card card = cardGroup.RemoveBack();            // remove last
Card card = cardGroup.RemoveAtPosition(2);      // remove at index
cardGroup.RemoveCard(card);                    // remove specific card

// With a hide animation:
cardGroup.RemoveFront(new FadeOutHideAnimation(), (removedCard) => {
    Destroy(removedCard.gameObject);
});
```

6.3. Moving Cards Between Groups

```
card.MoveToGroup(otherGroup);

// Or with more control:
sourceGroup.RemoveCard(card);
targetGroup.AddBack(card, new SwirlPlaceAnimation());
```

6.4. Shuffling

```
cardGroup.Shuffle();
```

7. Click Actions

Set `onClickAction` on the `CardGroup` :

Action	Behavior
Nothing	No click response.
Flip	Flips the card face-up or face-down.
Select	Toggles card selection.
Custom	Fires the <code>onClickCustom</code> UnityEvent.

8. Card Selection

Set `onClickAction = Select` on the `CardGroup` .

```
List<Card> selected    = cardGroup.GetSelectedCards();
List<Card> remaining  = cardGroup.GetNotSelectedCards();

bool isSelected = cardGroup.IsSelected(card);
```

Optionally enable `showSelectionMark` on the `CardGroup` to show or hide the `selectionMark` `GameObject` on each card.

9. Card Flipping

```
card.FlipCard();           // animated flip with Y-axis rotation
card.FlipCardNoAnimation(); // instant flip
```

Both methods toggle between the front and back faces. The card automatically detects rotation-based flips during animations.

10. Drag-and-Drop Between Groups

1. Set up a **source** `CardGroup` with `allowPickup = true`.
2. Add a `CardDropZone` component to a UI element.
3. Assign a **destination** `CardGroup` to the drop zone's `cardGroup` field.

The drop zone automatically shows when a card is being dragged and hides when the drag ends. When a card is dropped:

- **Place / Attack cards:** removed from the source group, added to the destination group, and the place animation plays.
- **Use cards:** the use animation plays and the card is destroyed (when `autoDestroyOnUse` is `true`).

Set `allowDroppingCardsFromDestinationGroup = true` if cards should be droppable back into the same group they came from.

11. Reordering Cards During Drag

Enabled by default (`allowReorderingDuringDrag = true` on `CardGroup`). When a player drags a card horizontally within its group, other cards slide aside and the card snaps into its new position on drop.

If the player drags the card too far vertically (beyond `reorderYThreshold`), reordering is suspended — useful so that dragging toward a drop zone does not accidentally reorder the hand.

12. Targeting Arrow

For Attack and TargettedUse cards, a dotted arrow automatically appears during drag. Set up the `DynamicArrow` component on your card prefab and assign `nodePrefab` (dot) and `headPrefab` (arrow tip).

Configure on `DynamicArrow` :

- **Distance Per Node** – spacing between dots.
- **Max Nodes** – pool limit.
- **First / Last Node Scale** – dots grow from start to end.
- **Head Scale** – arrow tip size.

13. Responding to Card Events

There are two ways to add game logic when cards are interacted with.

13.1. Option 1 — Subclass CardData

Best for logic that belongs to a specific card type (for example, "Fireball deals 5 damage on use").

```
[CreateAssetMenu(menuName = "MyGame/Fireball Card")]
public class FireballCardData : CardData
{
    public int damage = 5;

    public override void OnTargetedUse(Card card, Card target)
    {
        var health = target.GetComponent<CardHealth>();
        health?.TakeDamage(damage);
    }
}
```

Available overrides:

Method	When It Fires
<code>InitializeCard(Card)</code>	Card data is assigned via <code>Initialize()</code> .
<code>OnPickedUp(Card)</code>	Card is clicked (pointer down).
<code>OnDropped(Card)</code>	Drag ended without placing on a drop zone.
<code>OnPlaced(Card)</code>	Card placed onto a drop zone.
<code>OnTargetingStart(Card)</code>	Attack or TargettedUse drag started.

Method	When It Fires
<code>OnAttackHit(Card, Card)</code>	Attack animation reaches the target.
<code>OnUsed(Card, CardGroup)</code>	Use animation completed.
<code>OnTargetedUse(Card, Card)</code>	Targeted use animation completed.

Always call `base.InitializeCard(card)` when overriding `InitializeCard` — the base method sets the front and back art sprites.

13.2. Option 2 — Subscribe to Events

Best for external systems such as score tracking, sound effects, and UI updates.

```
card.onPickedUp      += () => audioSource.PlayOneShot(pickupSound);
card.onPlaced        += () => scoreManager.AddPoints(10);
card.onAttackHit     += (target) => SpawnHitVFX(target.transform.position);
card.onUsed          += (dropZone) => DrawCards(2);
card.onTargetedUse    += (target) => ApplyBuff(target);
card.onDropped       += () => Debug.Log("Card dropped");
card.onTargetingStarted += () => ShowTargetingUI();
card.onTargeted       += () => HighlightCard();
card.onUntargeted     += () => RemoveHighlight();
```

Both mechanisms fire together — the `CardData` override runs first, followed by the `Action` event.

14. Card Health

Add the `CardHealth` component alongside `Card` on your card prefab.

```
CardHealth health = card.GetComponent<CardHealth>();
health.Initialize(20); // set maximum health
health.TakeDamage(5);
health.Heal(3);
health.SetHealth(10); // set to an exact value
```

When health reaches zero the card is automatically removed from its group and destroyed.

Wire a UI `Text` element to the `healthDisplay` field to show current and maximum health, then call `health.StartDisplayingHealth()` to activate it.

Events:

```
health.OnHealthChanged += (int current, int max) => { };
health.OnDied           += () => { };
CardHealth.OnCardDied   += (CardHealth deadCard) => { }; // static – fires for any card
```

15. Dealing Cards with Delay

Use `CardGroupInitializer` with the `delay` field set to a positive value (in seconds). Cards will be added one at a time with that delay between each. Alternatively, write a coroutine:

```
IEnumerator DealCards(CardGroup source, CardGroup dest, int count, float delay)
{
    for (int i = 0; i < count; i++)
    {
        Card card = source.Back();
        card.MoveToGroup(dest);
        card.FlipCard();
        yield return new WaitForSeconds(delay);
    }
}
```

16. Tilt Effect

Cards gently wobble when idle and tilt toward the cursor on hover. Configured per `CardGroup`:

- **Enable Tilt** – toggle the whole effect.
- **Tilt Towards Cursor** – tilt toward or away from the mouse on hover.
- **Auto Tilt Amount** – idle wobble amplitude in degrees.
- **Manual Tilt Amount** – hover tilt amplitude in degrees.
- **Tilt Speed** – how fast the tilt interpolates.

17. Drag Spring

When dragging a Place or Use card, it wobbles based on mouse movement (like a spring). Configured per `Card`:

- **Enable Drag Spring** – toggle the effect.
- **Drag Spring Input Scale** – mouse delta sensitivity.
- **Drag Spring Max Angle** – maximum rotation angle.
- **Stiffness** – spring return speed (higher = snappier).

- **Damping** – how quickly oscillation settles.

18. Global Settings

Create a settings asset via **Create > CardControllerPro > Settings** and place it in a **Resources** folder so it loads automatically.

Setting	Default	Description
<code>useUnscaledTime</code>	<code>false</code>	When <code>true</code> , all card animations ignore <code>Time.timeScale</code> and keep animating while the game is paused.

19. Creating Custom Animations

You can create your own animation for any category. The following is a Place animation example:

```

using System;
using DG.Tweening;
using MackySoft.SerializeReferenceExtensions;
using UnityEngine;

[Serializable]
[AddTypeMenu("Bounce", 110)]
public class BouncePlaceAnimation : IPlaceAnimation
{
    [SerializeField] private float duration = 0.4f;
    [SerializeField] private float bounceStrength = 1.3f;

    public void Place(Card card, Vector3 target, Action onComplete)
    {
        card.isBeingPlaced = true;
        card.transform.DOKill();

        DOTween.Sequence()
            .Append(card.transform
                .DOScale(Vector3.one * bounceStrength, duration * 0.5f)
                .SetEase(Ease.OutQuad))
            .Append(card.transform
                .DOScale(Vector3.one, duration * 0.5f)
                .SetEase(Ease.InQuad))
            .SetUpdate(CardControllerSettings.Instance.useUnscaledTime)
            .OnComplete(() =>
            {
                card.isBeingPlaced = false;
                onComplete?.Invoke();
            });
    }
}

```

The same steps apply to any animation type:

1. Create a `[Serializable]` class implementing the appropriate interface.
2. Add `[AddTypeMenu("Name", order)]` so it appears in the Inspector dropdown.
3. Add `[SerializeField]` fields for any configurable parameters.
4. **Always call `onComplete`** when the animation finishes (for interfaces that provide a callback).
5. Call `.SetUpdate(CardControllerSettings.Instance.useUnscaledTime)` on all DOTween sequences and tweens.

The animation will appear automatically in the Inspector dropdown — no registration is required.

20. Script Reference

20.1. Card

MonoBehaviour. The core controller for a single card. Handles all Unity pointer events, drag logic, flip, tilt, targeting, and spring physics.

INSPECTOR FIELDS

Field	Type	Default	Purpose
<code>cardData</code>	CardData	null	ScriptableObject with card configuration and animations.
<code>cardTilter</code>	GameObject	–	Hierarchy node for tilt rotation.
<code>cardDragRotator</code>	GameObject	–	Hierarchy node for drag spring rotation.
<code>cardFlipper</code>	GameObject	–	Hierarchy node for Y-axis flip.
<code>cardHoverer</code>	GameObject	–	Hierarchy node for hover visual effects.
<code>cardPickuper</code>	GameObject	–	Hierarchy node for pickup visual effects.
<code>cardVisuals</code>	GameObject	–	Container for front, back, and border.
<code>front</code>	GameObject	–	Front face of the card.
<code>back</code>	GameObject	–	Back face of the card.
<code>border</code>	GameObject	–	Targeting border highlight.
<code>selectionMark</code>	GameObject	–	Selection indicator.
<code>canTargetOwnCardGroup</code>	bool	false	Allow targeting cards in the same group.
<code>flipDuration</code>	float	0.1	Half-flip animation duration in seconds.
<code>enableDragSpring</code>	bool	true	Enable spring physics during drag.
<code>dragSpringInputScale</code>	float	20	Mouse delta multiplier for spring input.
<code>dragSpringMaxAngle</code>	float	25	Maximum spring rotation angle in degrees.

KEY METHODS

Method	Description
<code>Initialize(CardData data)</code>	Assigns data and calls <code>InitializeCard</code> .
<code>MoveToGroup(CardGroup group)</code>	Removes the card from its current group and adds it to the new group.

Method	Description
<code>FlipCard()</code>	Animated Y-axis flip toggling front and back faces.
<code>FlipCardNoAnimation()</code>	Instant flip with no animation.
<code>ExecuteAttack(Transform target, Action onComplete)</code>	Plays the configured attack animation.
<code>ExecuteUse(CardGroup dropZoneGroup)</code>	Plays the use animation and destroys the card.
<code>ExecuteTargettedUse(Card target)</code>	Plays the targeted use animation and destroys the card.
<code>PlayPlaceAnimation()</code>	Plays the configured place animation.

INSTANCE EVENTS

Event	Signature	When It Fires
<code>onPickedUp</code>	<code>Action</code>	Pointer down on the card.
<code>onDropped</code>	<code>Action</code>	Drag ended without placement.
<code>onPlaced</code>	<code>Action</code>	Card placed on a drop zone.
<code>onTargetingStarted</code>	<code>Action</code>	Attack or targeting drag started.
<code>onAttackHit</code>	<code>Action<Card></code>	Attack impact moment.
<code>onUsed</code>	<code>Action<CardGroup></code>	Use animation completed.
<code>onTargetedUse</code>	<code>Action<Card></code>	Targeted use completed.
<code>onTargeted</code>	<code>Action</code>	This card became a target.
<code>onUntargeted</code>	<code>Action</code>	This card lost target status.

20.2. CardGroup

MonoBehaviour. Manages a list of `Card` objects with automatic layout and animation.

KEY INSPECTOR FIELDS

Field	Type	Default	Purpose
<code>cardSpacing</code>	float	1	Horizontal spacing between cards.
<code>fanRadius</code>	float	0	0 = straight line; nonzero = arc/fan layout.
<code>yOffsetPerCard</code>	float	0	Vertical stacking offset per card index.

Field	Type	Default	Purpose
<code>zOffsetPerCard</code>	float	0	Z stacking offset per card index.
<code>cardScale</code>	float	1	Scale applied to each card in this group.
<code>anchorToFirstCard</code>	bool	false	Position from card 0 instead of centering.
<code>tweenDuration</code>	float	0.3	Duration of position/rotation tweens.
<code>hoverYOffset</code>	float	0.5	Y lift on hover.
<code>makeRoomOnHover</code>	bool	true	Push neighboring cards aside on hover.
<code>hoverSpacing</code>	float	0.5	Distance neighbors shift when making room.
<code>hoverAnimation</code>	IHoverAnimation	ScaleUpHoverAnimation	Visual effect on hover.
<code>onClickAction</code>	OnClickAction	Nothing	What happens on card click.
<code>selectAnimation</code>	ISelectAnimation	LiftSelectAnimation	Visual effect on selection.
<code>allowPickup</code>	bool	true	Allow dragging cards from this group.
<code>allowReorderingDuringDrag</code>	bool	true	Allow reordering within the group during drag.
<code>reorderYThreshold</code>	float	200	Max Y distance for reordering to apply.
<code>isTargettable</code>	bool	true	Cards in this group can be attack targets.
<code>isPlayerHand</code>	bool	false	Marks the group as the player's hand.
<code>enableTilt</code>	bool	true	Enable the idle and hover tilt effect.

KEY METHODS

```
// Add
void AddFront(Card card, IPlaceAnimation placeAnimation = null)
void AddBack(Card card, IPlaceAnimation placeAnimation = null)
void AddAtPosition(Card card, int index, IPlaceAnimation placeAnimation = null)

// Access
Card Front()    // first card, or null
Card Back()     // last card, or null

// Remove
Card RemoveFront(IHideAnimation hideAnim = null, Action<Card> onComplete = null)
Card RemoveBack(IHideAnimation hideAnim = null, Action<Card> onComplete = null)
Card RemoveAtPosition(int index, IHideAnimation hideAnim = null, Action<Card> onComplete = null)
void RemoveCard(Card card, IHideAnimation hideAnim = null, Action<Card> onComplete = null)

// Selection
bool IsSelected(Card card)
List<Card> GetSelectedCards()
List<Card> GetNotSelectedCards()

// Other
void Shuffle()
void RecalculateAndAnimatePositions(Card skipAnimating = null)
```

20.3. CardData

ScriptableObject. Data container plus animation references and virtual lifecycle hooks. Subclass to add game-specific data and behavior. Create via **Assets > Create > CardControllerPro > Card Data**.

SERIALIZED FIELDS

Field	Type	Default	Purpose
<code>cardName</code>	string	–	Display name.
<code>description</code>	string	–	Card description.
<code>frontArt</code>	Sprite	null	Front face image.
<code>backArt</code>	Sprite	null	Back face image.
<code>cardType</code>	CardType	Place	Determines drag and interaction behavior.
<code>placeableFromHand</code>	bool	true	

Field	Type	Default	Purpose
			Override type to Place when in the player hand.
<code>autoDestroyOnUse</code>	bool	true	Auto-destroy after Use or TargettedUse animation.
<code>placeAnimation</code>	<code>IPlaceAnimation</code>	<code>DefaultPlaceAnimation</code>	Place animation strategy.
<code>attackAnimation</code>	<code>IAttackAnimation</code>	<code>ArcAttackAnimation</code>	Attack animation strategy.
<code>pickupAnimation</code>	<code>IPickupAnimation</code>	<code>DefaultPickupAnimation</code>	Pickup animation strategy.
<code>useAnimation</code>	<code>IUseAnimation</code>	<code>NoneUseAnimation</code>	Use animation strategy.

VIRTUAL LIFECYCLE METHODS

Method	When It Fires
<code>InitializeCard(Card card)</code>	Card data is assigned via <code>Initialize()</code> . Sets front/back sprites.
<code>OnPickedUp(Card card)</code>	Pointer down on the card.
<code>OnDropped(Card card)</code>	Drag ended without placing on a drop zone.
<code>OnPlaced(Card card)</code>	Card placed on a drop zone.
<code>OnTargetingStart(Card card)</code>	Attack or TargettedUse drag started.
<code>OnAttackHit(Card attacker, Card target)</code>	Attack animation reaches the target.
<code>OnUsed(Card card, CardGroup target)</code>	Use animation completed.
<code>OnTargetedUse(Card card, Card target)</code>	Targeted use animation completed.

20.4. CardDropZone

MonoBehaviour. Receives dragged cards. Activates and deactivates automatically when `Card.OnCardMoved` fires.

Field	Type	Default	Purpose
<code>activateWhenMovingCards</code>	bool	true	Auto-show when a card is being dragged.
<code>cardGroup</code>	<code>CardGroup</code>	–	Target group to add dropped cards to.
<code>allowDroppingCardsFromDestinationGroup</code>	bool	false	Allow dropping cards already in the target group.

20.5. CardGroupInitializer

MonoBehaviour. Spawns cards into groups on `Start`. Configure entirely in the Inspector.

```
[Serializable]
public struct CardEntry {
    public GameObject cardPrefab; // must have Card component
    public CardData   cardData;
    public int        count;      // number of copies (minimum 1)
}

[Serializable]
public struct CardGroupEntry {
    public CardGroup      cardGroup;
    public List<CardEntry> cards;
    public bool           shuffle;    // shuffle after all cards are added
    public bool           flipped;    // flip cards face-down
    public float          delay;      // seconds between cards (0 = instant)
    public IPlaceAnimation placeAnimation; // per-card animation
}
```

20.6. CardHealth

MonoBehaviour. Runtime health tracking with automatic removal and destruction on death.

Member	Kind	Description
<code>Initialize(int maxHp)</code>	Method	Set maximum health.
<code>TakeDamage(int amount)</code>	Method	Reduce health by amount.
<code>Heal(int amount)</code>	Method	Increase health by amount.
<code>SetHealth(int value)</code>	Method	Set health to an exact value.
<code>StartDisplayingHealth()</code>	Method	Show the health display UI element.
<code>OnHealthChanged</code>	Event <code>Action<int,int></code>	Fires when health changes; parameters are current and maximum health.
<code>OnDied</code>	Event <code>Action</code>	Fires when this card's health reaches zero.
<code>CardHealth.OnCardDied</code>	Static event <code>Action<CardHealth></code>	Fires for any card death globally.

20.7. CardControllerSettings

ScriptableObject singleton loaded from a `Resources` folder. Create via **Assets > Create > CardControllerPro > Settings**.

Field	Type	Default	Description
<code>useUnscaledTime</code>	bool	false	When true, all DOTween animations use unscaled time and continue while the game is paused.

Access via `CardControllerSettings.Instance.useUnscaledTime`.

20.8. Animation Interfaces

Interface	Configured On	Method Signatures
<code>IPlaceAnimation</code>	CardData	<code>Place(Card card, Vector3 target, Action onComplete)</code>
<code>IAttackAnimation</code>	CardData	<code>Attack(Card attacker, Transform target, Action onAttack, Action onComplete)</code>
<code>IHoverAnimation</code>	CardGroup	<code>Hover(Card card) / Unhover(Card card)</code>
<code>IPickupAnimation</code>	CardData	<code>Pickup(Card card) / PutDown(Card card)</code>
<code>ISelectAnimation</code>	CardGroup	<code>Select(Card card) / Deselect(Card card)</code>
<code>IUseAnimation</code>	CardData	<code>Use(Card card, Transform target, Action onComplete)</code>
<code>IHideAnimation</code>	Remove parameter	<code>Hide(Card card, Action onComplete)</code>

21. Creating an Assembly Definition

If you want to create an Assembly Definition (`.asmdef`) for Card Controller Pro:

1. Open the **DOTween Utility Panel** (*Tools > Demigiant > DOTween Utility Panel*) and click the **Create ASMDEF** button, located just below the Setup button.
2. In the `CardControllerPro` directory, create an **Assembly Definition Reference** asset and add the following assembly references to it:
 - `MackySoft.SerializeReferenceExtensions`
 - `MackySoft.SerializeReferenceExtensions.Editor`
 - `Unity.InputSystem`
 - The DOTween assembly created in step 1.
3. Click **Apply**.

22. Included Prefabs

Prefab	Description
<code>Card.prefab</code>	Standard card with the full hierarchy.

Prefab	Description
<code>TCGCard.prefab</code>	TCG-styled variant.
<code>BalatroCard.prefab</code>	Balatro-styled variant with the <code>BalatroCardValue</code> component.
<code>Node.prefab</code>	Dot for the targeting arrow.
<code>Head.prefab</code>	Arrow tip for the targeting arrow.

23. Demo Scenes

The included demo scenes show different use cases. The demo scripts (`DemoCardDealer` , `PokerDemoCardDealer` , `CardConfirmerDemo` , `CardTester`) are for demonstration purposes only. Use `CardGroupInitializer` for production card spawning.